# Customer Shopping Behavior Analysis

Python • SQL • Power BI Dashboard

*Full End-to-End Data Analysis Project — 3,900 Transactions*

**Uncovering insights into spending patterns, customer segments, product preferences, and subscription behavior.**

**Prepared by: Hamza Raed Alhadidi**

**Date: March 2025**

**Dataset: 3,900 Transactions | 50 US States | 25 Products**

# 1. Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases at a retail store operating across 50 US states, covering both in-store and online channels.

| | | |
|---|---|---|
| **Tool 1** | **Python + Pandas** | Data loading, cleaning, feature engineering, and export to SQL Server |
| **Tool 2** | **SQL Server** | Business queries — 13 questions answered using T-SQL |
| **Tool 3** | **Power BI** | 4-page interactive dashboard with DAX measures and visual storytelling |

# 2. Dataset Summary

| Property | Value |
|---|---|
| Rows | 3,900 |
| Columns | 18 (original) → 20 (after feature engineering) |
| Missing Data | 37 values in Review Rating only |
| Duplicates | 0 |
| Age Range | 18 – 70 years |
| Purchase Range | $20 – $100 |
| Locations | 50 US States |

### Key Features

| | | |
|---|---|---|
| **Demo** | **Customer Demographics** | Age, Gender, Location, Subscription Status |
| **Purch** | **Purchase Details** | Item Purchased, Category, Purchase Amount, Season, Size, Color |
| **Behav** | **Shopping Behavior** | Discount Applied, Previous Purchases, Frequency, Review Rating, Shipping Type |
| **Eng** | **Engineered Features** | age_group (binned), purchases_frequency_days (mapped), loyalty_tier |

# 3. Exploratory Data Analysis using Python

> *Full notebook: Customer_Shopping_EDA.ipynb — All steps documented with comments*

## 3.1 Data Loading & Initial Exploration

```
import pandas as pd
import numpy as np

df = pd.read_csv('customer_shopping_behavior.csv')
df.info()      # Check structure and data types
df.describe(include='all')   # Summary statistics
```

The dataset contains 3,900 rows and 18 columns. All columns are complete except Review Rating which has 37 missing values (0.95%).

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

```
df.describe(include='all')
```

| | Customer ID | Age | Gender | Item Purchased | Category | Purchase Amount (USD) | Location | Size | Color | Season | Review Rating | Subscription Status | Shipping Type | Discount Applied | Promo Code Used | Previous Purchases | Payment Method | Frequency of Purchases |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **count** | 3900.000000 | 3900.000000 | 3900 | 3900 | 3900 | 3900.000000 | 3900 | 3900 | 3900 | 3900 | 3863.000000 | 3900 | 3900 | 3900 | 3900 | 3900.000000 | 3900 | 3900 |
| **unique** | NaN | NaN | 2 | 25 | 4 | NaN | 50 | 4 | 25 | 4 | NaN | 2 | 6 | 2 | 2 | NaN | 6 | 7 |
| **top** | NaN | NaN | Male | Blouse | Clothing | NaN | Montana | M | Olive | Spring | NaN | No | Free Shipping | No | No | NaN | PayPal | Every 3 Months |
| **freq** | NaN | NaN | 2652 | 171 | 1737 | NaN | 96 | 1755 | 177 | 999 | NaN | 2847 | 675 | 2223 | 2223 | NaN | 677 | 584 |
| **mean** | 1950.500000 | 44.068462 | NaN | NaN | NaN | 59.764359 | NaN | NaN | NaN | NaN | 3.750065 | NaN | NaN | NaN | NaN | 25.351538 | NaN | NaN |
| **std** | 1125.977353 | 15.207589 | NaN | NaN | NaN | 23.685392 | NaN | NaN | NaN | NaN | 0.716983 | NaN | NaN | NaN | NaN | 14.447125 | NaN | NaN |
| **min** | 1.000000 | 18.000000 | NaN | NaN | NaN | 20.000000 | NaN | NaN | NaN | NaN | 2.500000 | NaN | NaN | NaN | NaN | 1.000000 | NaN | NaN |
| **25%** | 975.750000 | 31.000000 | NaN | NaN | NaN | 39.000000 | NaN | NaN | NaN | NaN | 3.100000 | NaN | NaN | NaN | NaN | 13.000000 | NaN | NaN |
| **50%** | 1950.500000 | 44.000000 | NaN | NaN | NaN | 60.000000 | NaN | NaN | NaN | NaN | 3.800000 | NaN | NaN | NaN | NaN | 25.000000 | NaN | NaN |
| **75%** | 2925.250000 | 57.000000 | NaN | NaN | NaN | 81.000000 | NaN | NaN | NaN | NaN | 4.400000 | NaN | NaN | NaN | NaN | 38.000000 | NaN | NaN |
| **max** | 3900.000000 | 70.000000 | NaN | NaN | NaN | 100.000000 | NaN | NaN | NaN | NaN | 5.000000 | NaN | NaN | NaN | NaN | 50.000000 | NaN | NaN |

## 3.2 Missing Data Handling

Only one column had missing values — Review Rating (37 nulls). Filled using the median of each product category rather than the overall median, because different categories have different rating distributions.

```
# Fill missing Review Rating with median of each Category
df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(
    lambda x: x.fillna(x.median())   # Note: median() with parentheses
)
```

## 3.3 Data Consistency Check

Verified that Discount Applied and Promo Code Used are 100% identical across all 3,900 rows — confirmed redundancy and dropped Promo Code Used.

```
# Verify both columns are identical
(df['Discount Applied'] == df['Promo Code Used']).all()  # True

# Drop redundant column
df = df.drop(columns=['Promo Code Used'])
```

## 3.4 Column Standardization

```
# Convert to snake_case for SQL compatibility
df.columns = (
    df.columns.str.lower().str.strip()
    .str.replace(' ', '_').str.replace(r'[^a-z0-9_]', '', regex=True)
)
df = df.rename(columns={'purchase_amount_usd': 'purchase_amount'})
```

## 3.5 Feature Engineering

```
# 1. Age Group — binning customers into 4 age segments
bins   = [18, 30, 45, 60, 70]
labels = ['Young Adult', 'Adult', 'Middle-Aged', 'Senior']
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels,
                         right=True, include_lowest=True).astype(str)

# 2. Purchase Frequency in Days — standardize category names
df['frequency_of_purchases'] =
df['frequency_of_purchases'].replace('Fortnightly','Bi-Weekly')

frequency_mapping = {
    'Weekly': 7,  'Bi-Weekly': 14, 'Monthly': 30,
    'Quarterly': 90, 'Every 3 Months': 90, 'Annually': 365
}
df['purchases_frequency_days'] =
df['frequency_of_purchases'].map(frequency_mapping)
```

| age_group | Count |
|---|---|
| Middle-Aged | 1135 |
| Adult | 1,093 |
| Young Adult | 949 |
| Senior | 723 |

## 3.6 Database Export

```
from sqlalchemy import create_engine
from urllib.parse import quote_plus

server   = r'localhost\SQLEXPRESS'
database = 'customer_behavior'
driver   = quote_plus('ODBC Driver 17 for SQL Server')

engine = create_engine(

f'mssql+pyodbc://@{server}/{database}?driver={driver}&trusted_connection=yes',
    fast_executemany=True
)

df.to_sql('customer', engine, if_exists='replace', index=False, chunksize=500)
```

# 4. Data Analysis using SQL

> *All queries run on SQL Server against the customer table. Results replicated from the actual dataset.*

## Q1 Total Revenue by Gender

```
SELECT gender AS Gender,
       SUM(CAST(purchase_amount AS Float)) AS Revenue
FROM customer
GROUP BY gender;
```

| Gender | Revenue |
|---|---|
| Female | $75,191 |
| Male | $157,890 |

Male customers account for 68% of total revenue ($157,890 vs $75,191), reflecting the gender imbalance in the dataset (68% male).

## Q2 High-Spending Discount Users

```
SELECT customer_id, purchase_amount
FROM customer
WHERE discount_applied = 'Yes'
  AND purchase_amount > (SELECT AVG(CAST(purchase_amount AS Float)) FROM
customer);
```

| customer_id | purchase_amount | Note |
|---|---|---|
| 2 | $64 | Above $59.76 avg |
| 3 | $73 | Above avg |
| 4 | $90 | Above avg |
| 7 | $85 | Above avg |
| 9 | $97 | Above avg |
| ... | ... | 839 total rows |

839 customers (21.5%) used a discount AND spent above the average — these are the most valuable discount users.

## Q3 Top 5 Products by Avg Review Rating

```
SELECT TOP 5 item_purchased,
       ROUND(AVG(review_rating), 2) AS avg_review_rating
FROM customer
GROUP BY item_purchased
ORDER BY avg_review_rating DESC;
```

| item_purchased | avg_review_rating |
|---|---|
| Gloves | 3.86 |
| Sandals | 3.84 |
| Boots | 3.82 |
| Hat | 3.80 |
| Handbag | 3.78 |

## Q4 Standard vs Express Shipping — Avg Purchase

```
SELECT shipping_type,
       ROUND(AVG(CAST(purchase_amount AS Float)), 2) AS avg_purchase
FROM customer
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type;
```

| shipping_type | avg_purchase |
|---|---|
| Express | $60.48 |
| Standard | $58.46 |

Express shipping customers spend ~$2 more on average. The difference is small, suggesting shipping type doesn't strongly influence purchase amount.

## Q5 Subscribers vs Non-Subscribers

```
SELECT subscription_status,
       COUNT(customer_id)                              AS total_customers,
       ROUND(AVG(CAST(purchase_amount AS Float)), 2)   AS avg_spend,
       SUM(CAST(purchase_amount AS Float))             AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue DESC;
```

| subscription_status | total_customers | avg_spend | total_revenue |
|---|---|---|---|
| No | 2,847 | $59.87 | $170,436 |
| Yes | 1,053 | $59.49 | $62,645 |

Surprisingly, non-subscribers have a slightly higher avg spend ($59.87 vs $59.49). However, they also represent 73% of customers — subscription promotion could increase LTV.

## Q6 Top 5 Products by Discount Rate

```
SELECT TOP 5 item_purchased,
       CAST(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0
END)
            / COUNT(*) AS DECIMAL(5,2)) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC;
```

| item_purchased | discount_rate |
|---|---|
| Hat | 50.00% |
| Sneakers | 49.66% |
| Coat | 49.07% |
| Sweater | 48.17% |
| Pants | 47.37% |

## Q7 Customer Segmentation — New / Returning / Loyal

```
WITH customer_types AS (
    SELECT customer_id, previous_purchases,
           CASE
               WHEN previous_purchases <= 5  THEN 'New'
               WHEN previous_purchases <= 20 THEN 'Returning'
               ELSE 'Loyal'
           END AS customer_segment
    FROM customer
)
SELECT customer_segment, COUNT(*) AS number_of_customers
FROM customer_types
GROUP BY customer_segment
ORDER BY number_of_customers DESC;
```

| customer_segment | number_of_customers |
|---|---|
| Loyal | 3,116 (79.89%) |
| Returning | 360 (9.23%) |
| New | 424 (10.87%) |

The majority (80%) are Loyal customers, indicating strong retention. New customers represent only 11% — acquisition is lower priority than retention.

## Q8 Top 3 Products per Category

```
WITH item_counts AS (
    SELECT category, item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER(PARTITION BY category ORDER BY
COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank, category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <= 3;
```

| item_rank | category | item_purchased | total_orders |
|---|---|---|---|
| 1 | Accessories | Jewelry | 171 |
| 2 | Accessories | Belt | 161 |
| 3 | Accessories | Sunglasses | 161 |
| 1 | Clothing | Blouse | 171 |
| 2 | Clothing | Pants | 171 |
| 3 | Clothing | Shirt | 169 |
| 1 | Footwear | Sandals | 160 |
| 2 | Footwear | Shoes | 150 |
| 3 | Footwear | Sneakers | 145 |
| 1 | Outerwear | Jacket | 163 |
| 2 | Outerwear | Coat | 161 |

## Q9 Repeat Buyers & Subscription Rate

```
SELECT subscription_status,
       COUNT(*) AS repeat_buyers,
       CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER() AS DECIMAL(5,2)) AS pct
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

| subscription_status | repeat_buyers | pct |
|---|---|---|
| No | 2,518 | 72.44% |
| Yes | 958 | 27.56% |

Among customers with >5 purchases, only 27.56% subscribe — nearly identical to the overall subscription rate. Repeat buying does not strongly predict subscription.

## Q10 Revenue Contribution by Age Group

```
SELECT age_group, SUM(CAST(purchase_amount AS Float)) AS revenue
FROM customer
GROUP BY age_group
ORDER BY
    CASE age_group
        WHEN 'Young Adult' THEN 1  WHEN 'Adult'       THEN 2
        WHEN 'Middle-Aged' THEN 3  WHEN 'Senior'      THEN 4
    END;
```

| age_group | revenue |
|---|---|
| Young Adult | $57,279 |
| Adult | $64,927 |
| Middle-Aged | $67,711 |
| Senior | $43,164 |

Middle-Aged (46-60) generates the highest total revenue ($67,711) due to their larger customer count (1,135). All age groups spend nearly the same per transaction (range: $59.40 – $60.36), suggesting age does not significantly influence individual purchase amount.

## Q11 Top 10 Customers by Estimated Lifetime Spend

```
-- Part 1: Average estimated lifetime spend
SELECT ROUND(AVG(CAST(purchase_amount AS FLOAT) * previous_purchases), 2)
       AS avg_estimated_lifetime_spend
FROM customer;
-- Result: $1,517.88

-- Part 2: Top 10 customers
WITH customer_spending AS (
    SELECT customer_id, age_group,
           purchase_amount AS last_purchase_amount,
           previous_purchases AS total_orders,
           purchase_amount * previous_purchases AS estimated_lifetime_spend,
           CAST(purchase_amount AS FLOAT) / NULLIF(previous_purchases, 0) AS
avg_order_value
    FROM customer
)
SELECT TOP 10 customer_id, age_group, total_orders, last_purchase_amount,
       estimated_lifetime_spend,
       CAST(100.0 * estimated_lifetime_spend / SUM(estimated_lifetime_spend)
OVER()
            AS DECIMAL(5,2)) AS revenue_pct_of_total
FROM customer_spending
ORDER BY estimated_lifetime_spend DESC;
```

Average estimated lifetime spend per customer: $1,517.88

| customer_id | age_group | total_orders | last_purchase | avg_order_value | estimated_lifetime_spend | revenue_pct |
|---|---|---|---|---|---|---|
| 456 | Middle-Aged | 50 | $100 | 100 | $5,000 | 0.08% |
| 993 | Middle-Aged | 50 | $99 | 99 | $4,950 | 0.08% |
| 1738 | Adult | 50 | $98 | 98 | $4,900 | 0.08% |
| 1848 | Middle-Aged | 49 | $100 | 100 | $4,900 | 0.08% |
| 2272 | Young Adult | 49 | $99 | 99 | $4,851 | 0.08% |
| 886 | Adult | 49 | $99 | 99 | $4,851 | 0.08% |
| 3747 | Middle-Aged | 49 | $99 | 99 | $4,851 | 0.08% |
| 2485 | Middle-Aged | 50 | $97 | 97 | $4,850 | 0.08% |
| 2230 | Senior | 50 | $97 | 97 | $4,850 | 0.08% |
| 1830 | Young Adult | 50 | $96 | 96 | $4,800 | 0.08% |

## Q12 Best-Selling Product per State

```
WITH product_sales AS (
    SELECT location, item_purchased, COUNT(*) AS customers_count,
           ROW_NUMBER() OVER(PARTITION BY location ORDER BY COUNT(*) DESC)
AS rank_num
    FROM customer
    GROUP BY location, item_purchased
)
SELECT location, item_purchased, customers_count
FROM product_sales
WHERE rank_num = 1
ORDER BY location;
```

| location | item_purchased | customers_count |
|---|---|---|
| Alabama | Jewelry | 8 |
| Alaska | Backpack | 5 |
| Arizona | Backpack | 5 |
| Arkansas | Gloves | 6 |
| California | Dress | 7 |
| Colorado | Sneakers | 6 |
| Connecticut | Coat | 6 |
| Delaware | Pants | 7 |
| Florida | Hat | 6 |
| Georgia | Dress | 8 |
| ... | ... | 50 states total |

## Q13 Top 10 Customers Above Age Group Average

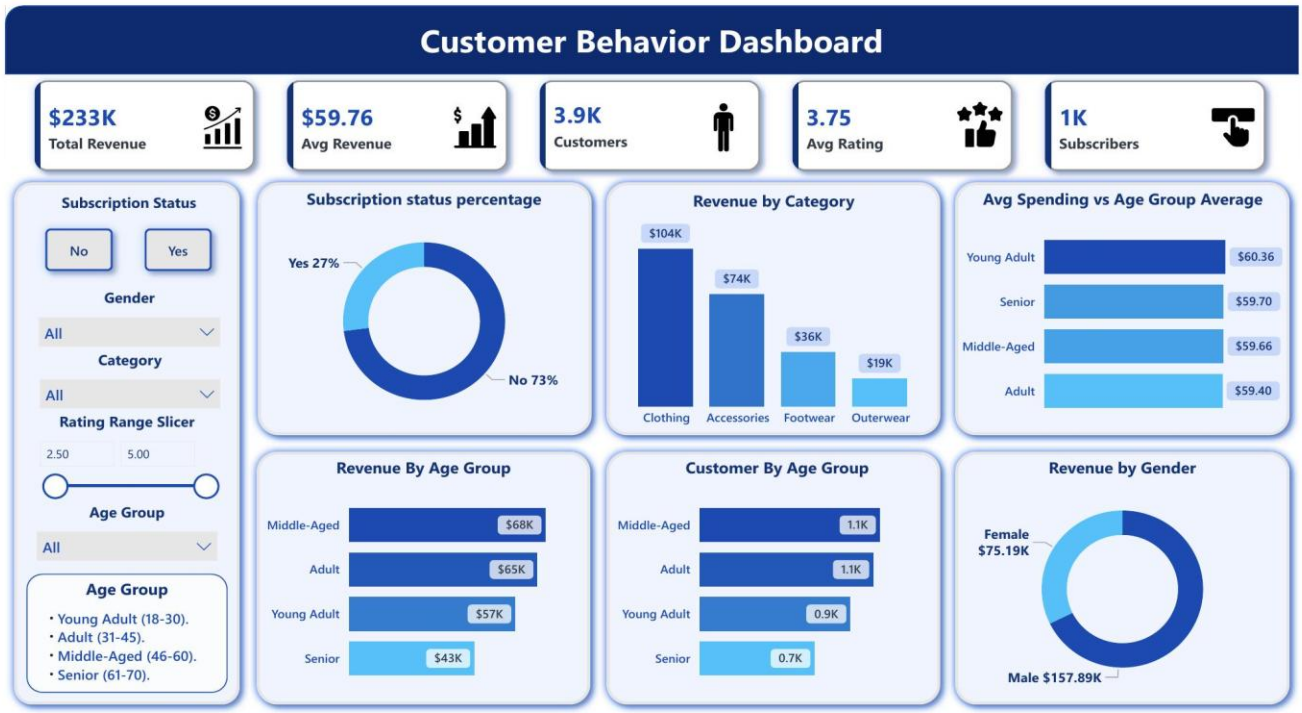
```
WITH age_analysis AS (
    SELECT customer_id, age_group, purchase_amount,
           ROUND(AVG(CAST(purchase_amount AS Float)) OVER(PARTITION BY
age_group), 2)
           AS age_group_avg
    FROM customer
),
ranked_customers AS (
    SELECT customer_id, age_group, purchase_amount, age_group_avg,
           ROUND(purchase_amount - age_group_avg, 2) AS above_avg,
           ROW_NUMBER() OVER(PARTITION BY age_group
               ORDER BY purchase_amount - age_group_avg DESC) AS
rank_in_group
    FROM age_analysis
    WHERE purchase_amount > age_group_avg
)
SELECT customer_id, age_group, purchase_amount, age_group_avg, above_avg
FROM ranked_customers
WHERE rank_in_group <= 10
ORDER BY age_group, rank_in_group;
```

| customer_id | age_group | purchase_amount | age_group_avg | above_avg |
|---|---|---|---|---|
| 96 | Adult | $100 | $59.40 | $40.60 |
| 194 | Adult | $100 | $59.40 | $40.60 |
| ... | Adult | ... | $59.40 | ... |
| 2959 | Middle-Aged | $100 | $59.66 | $40.34 |
| 2808 | Middle-Aged | $100 | $59.66 | $40.34 |
| ... | Middle-Aged | ... | $59.66 | ... |
| 616 | Senior | $100 | $59.70 | $40.30 |
| ... | Senior | ... | $59.70 | ... |
| 3726 | Young Adult | $100 | $60.36 | $39.64 |
| ... | Young Adult | ... | $60.36 | ... |

# 5. Dashboard in Power BI

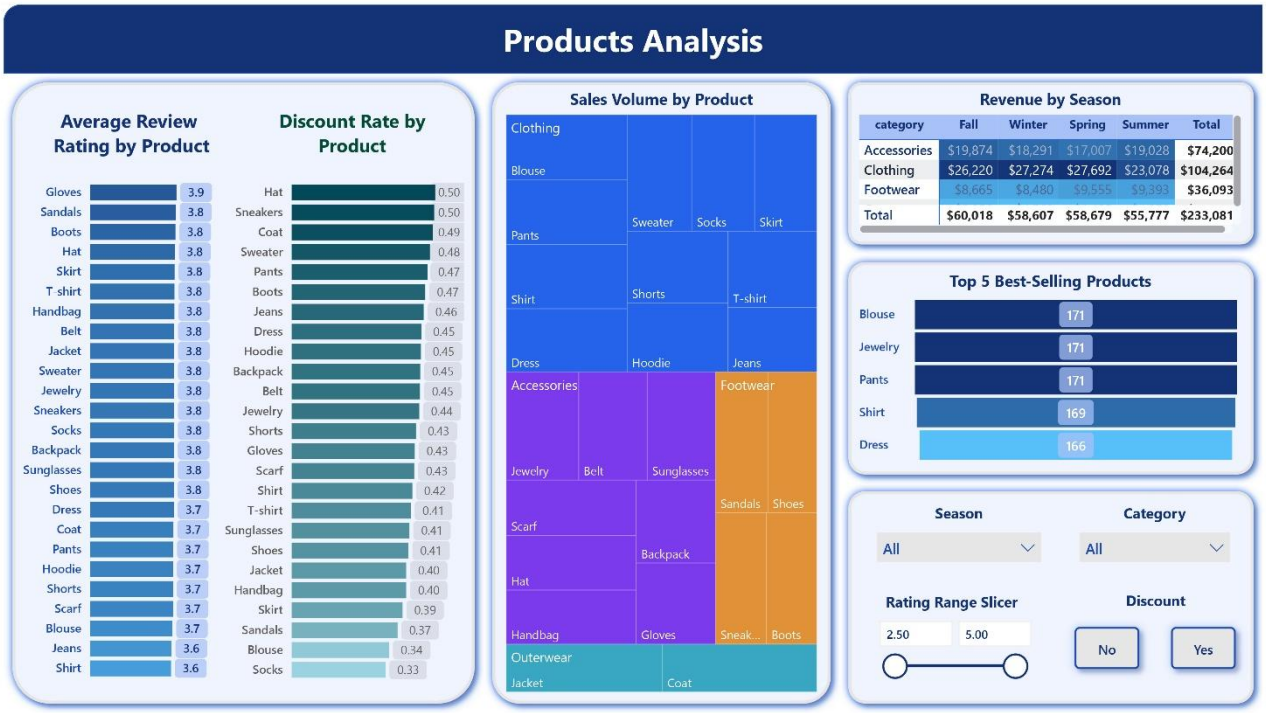
An interactive 4-page dashboard was built in Power BI to visualize all 13 SQL findings. The dashboard uses a unified blue theme (#0D2B6E header, #2563EB primary) with conditional formatting, DAX measures, and cross-page filters.

**Page 1 — Overview (Answers: Q1, Q2, Q5, Q10, Q13)**




| Visual | SQL Question Answered |
|---|---|
| Revenue by Gender (Donut) | Q1 — Male $157K vs Female $75K |
| KPI Cards | Q5 — Total Revenue $233K, Subscribers 1K (27%) |
| Revenue by Age Group (Bar) | Q10 — Middle-Aged leads at $68K |
| Avg Spending vs Age Group Avg | Q13 — Young Adult highest avg $60.36 |
| Subscription Status (Donut) | Q5 — Yes 27% / No 73% |
| Revenue by Category | Q1 — Clothing highest revenue $104 k |

## Page 2 — Products Analysis (Answers: Q3, Q6, Q8)

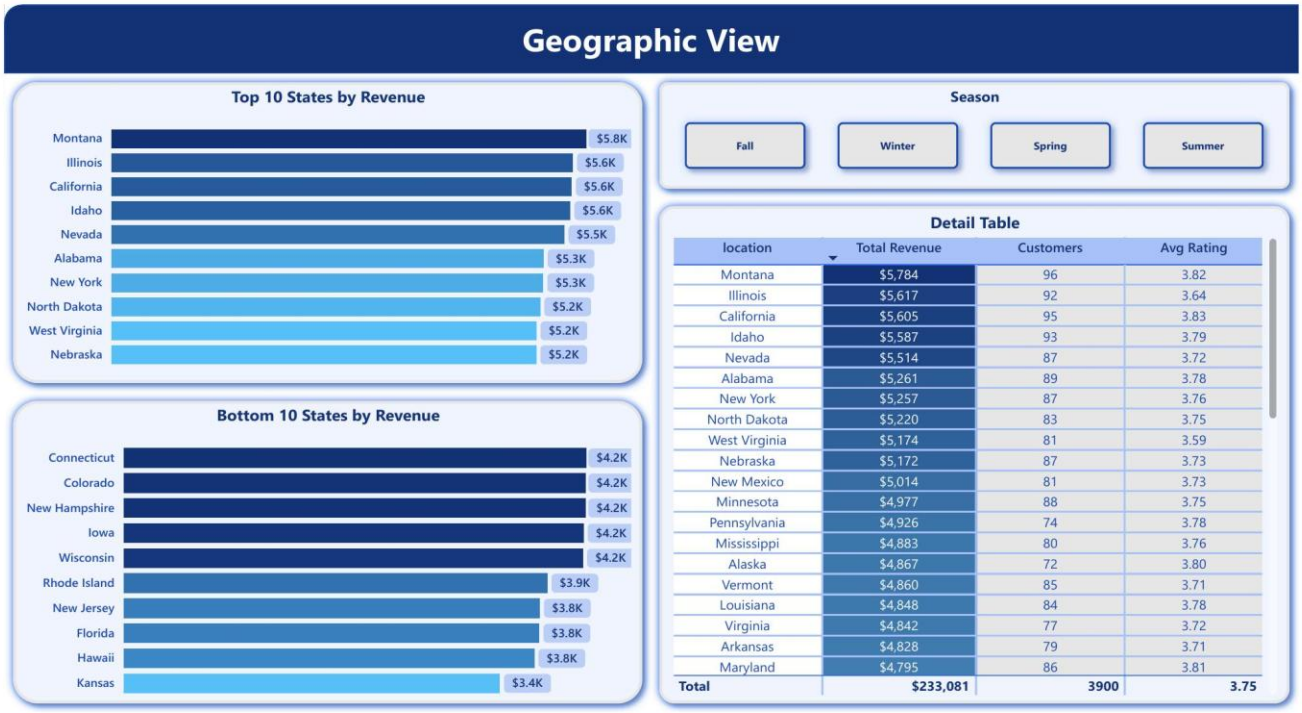



| Visual | SQL Question Answered |
|---|---|
| Avg Review Rating by Product | Q3 — Gloves 3.9, Sandals 3.84, Boots 3.82 |
| Discount Rate by Product | Q6 — Hat 50%, Sneakers 50%, Coat 49.07% |
| Revenue by Season (Matrix) | Q8 — Clothing Spring $27,692 highest cell |
| Top 5 Best-Selling Products | Q8 — Blouse, Jewelry, Pants all at 171 |
| Sales Volume Treemap | Q8 — Clothing dominates with 4 largest blocks |

## Page 3 — Customer Segments (Answers: Q4, Q7, Q9)

| Visual | SQL Question Answered |
|---|---|
| Customer Segments by Age Group | Q7 — Loyal 3,116 / Returning 360 / New 424 |
| Loyalty vs Spending Behavior | Q7 — Loyal buy 30 but spend less ($59.50) |
| Repeat Buyers (Bar) | Q9 — 72.44% No subscription, 27.56% Yes |
| Shipping Type vs Avg Revenue | Q4 — Express $60.48 vs Standard $58.46 |
| Payment Method (Donut) | PayPal leads at 17.36% |

## Page 4 — Geographic View (Answers: Q12)




| location | Total Revenue | Customers | Avg Rating |
|---|---|---|---|
| Montana | $5,784 | 96 | 3.82 |
| Illinois | $5,617 | 92 | 3.64 |
| California | $5,605 | 95 | 3.83 |
| Idaho | $5,587 | 93 | 3.79 |
| Nevada | $5,514 | 87 | 3.72 |
| Alabama | $5,261 | 89 | 3.78 |
| New York | $5,257 | 87 | 3.76 |
| North Dakota | $5,220 | 83 | 3.75 |
| West Virginia | $5,174 | 81 | 3.59 |
| Nebraska | $5,172 | 87 | 3.73 |
| New Mexico | $5,014 | 81 | 3.73 |
| Minnesota | $4,977 | 88 | 3.75 |
| Pennsylvania | $4,926 | 74 | 3.78 |
| Mississippi | $4,883 | 80 | 3.76 |
| Alaska | $4,867 | 72 | 3.80 |
| Vermont | $4,860 | 85 | 3.71 |
| Louisiana | $4,848 | 84 | 3.78 |
| Virginia | $4,842 | 77 | 3.72 |
| Arkansas | $4,828 | 79 | 3.71 |
| Maryland | $4,795 | 86 | 3.81 |
| **Total** | **$233,081** | **3900** | **3.75** |

| Visual | SQL Question Answered |
|---|---|
| Top 10 States by Revenue | Q12 — Montana $5,784 / Kansas lowest $3,437 |
| Bottom 10 States | Geographic revenue gap: $5,784 – $3,437 |
| Detail Table | All 50 states with Revenue, Customers, Avg Rating |
| Season Slicer | Filter any chart by Fall/Winter/Spring/Summer |

# 6. Key Findings

| | | |
|---|---|---|
| **F1** | **Revenue Imbalance** | Male customers generate 68% of total revenue ($157,890) vs Female ($75,191) — reflecting a 68/32 gender split in the dataset |
| **F2** | **Loyal Majority** | 80% of customers (3,116) are Loyal with 30+ purchases — retention programs matter more than acquisition |
| **F3** | **Discount Paradox** | 839 customers (21.5%) used discounts AND spent above average — discounts attract high-value buyers, not just bargain hunters |
| **F4** | **Shipping Neutral** | Max $2.27 difference in avg spend across all 6 shipping types — shipping choice does not drive purchase amount |
| **F5** | **Subscription Gap** | Only 27% subscribe despite 80% being Loyal — major opportunity to convert loyal buyers into subscribers |
| **F6** | **Age Insight** | Middle-Aged (46–60) generates highest total revenue ($67,711) but Young Adults spend most per transaction ($60.36 avg) |
| **F7** | **Rating Tight** | All 25 products rated between 3.6–3.9 — no standout product quality issue and no exceptional performer |
| **F8** | **Geographic Even** | Only $2.3K difference between top state (Montana $5,784) and bottom (Kansas $3,437) — no dominant market |

# 7. Business Recommendations

| | | |
|---|---|---|
| **R1** | **Boost Subscriptions** | Only 27% of customers subscribe despite 80% being Loyal. Launch a loyalty-to-subscription conversion campaign targeting the 2,324 Loyal non-subscribers. Even a 10% conversion rate adds ~232 new subscribers. |
| **R2** | **Smart Discount Strategy** | 839 customers used discounts AND spent above average — these are your best discount users. Target them with exclusive early-access offers rather than blanket discounts to protect margins. |
| **R3** | **Middle-Aged Marketing** | Middle-Aged customers (46–60) generate $67,711 in revenue — the highest of any group. Prioritize this segment in product placement, email campaigns, and loyalty rewards. |
| **R4** | **Promote Top Products** | Hat (50% discount rate) and Sneakers (49.66%) are heavily discount-dependent. Test raising their base price slightly to reduce discount dependency while maintaining volume. |
| **R5** | **Senior Engagement** | Senior customers (61–70) have the lowest revenue ($43,164) and smallest count (723). A targeted campaign with simplified UX and senior-friendly offers could unlock this underserved segment. |